



نظري

4

15

1800 sp

كلية الهندسة المعلوماتية

السنة الخامسة - هندسة البرمجيات

Behavioral Pattern & Petri Net

د. باسم قصيبة



RB Informatics; 04/11/2024

هندسة برمجيات ٣

تحدثنا في المحاضرة السابقة عن نوعين من Behavioral Pattern وهما:
Chain of responsibility و Template method.

وسنكمل في هذه المحاضرة بقية ال Behavioral Patterns ال Petri net ب نبدأ ب

قلنا سابقاً بأن chain of response هو مجموعة من الأغراض التي تعتمد على بعضها

فمثلاً:

لدينا طالب نجح في الثانوية العامة وقام بالمفاضلة على اختصاص السنة التحضيرية
أولاً نقوم بمطابقة مجموعه الصافي مع الحد الأدنى للقبول:
إذا كان مجموعه أكبر يدخل المفاضلة.
وإذا كان يساويه ننظر إلى علاماته في مواد الرياضيات والعلوم وهكذا.
نلاحظ عند تساوي الحد الأدنى لمعدل القبول الجامعي مع مجموع الطالب قمنا بالاعتماد على أغراض
أخرى (علاماته في العلوم وغيرها..).

أما template method عبارة عن class أساسي ومجموعة كلاسات ترث منه لتعديل سلوك التوابع (method)
الموجودة في ال class الأساسي ضمن هذه ال (sub classes) classes.

مثال: مسألة نقل البضائع باستخدام template Method

نقوم بعمل حاويات لنقل البضائع، كل بضاعة لها حجم ووزن وعند عمل حاوية container نضع فيها مواد حسب نوع
النقل.

إذا كان بري نركز على الوزن.

وإذا كان بحري أيضاً نركز على الوزن.

أما في حال كان جوي فهو يركز على الحجم.

ولدينا جدول يوضح الكمية و الكلفة وفقاً لنوع النقل.

فمثلاً النقل البحري تستطيع وضع كمية تصل ل 30 ألف كغ ونركز على الوزن... إلخ.



هذه المسألة هي مسألة نموذجية للتدرب على القيام بعمل برامج غرضية التوجه. برامج غرضية التوجه وُجدت لتجميع المفاهيم المشتركة (عامة) common concept وهذا هدف ال template.

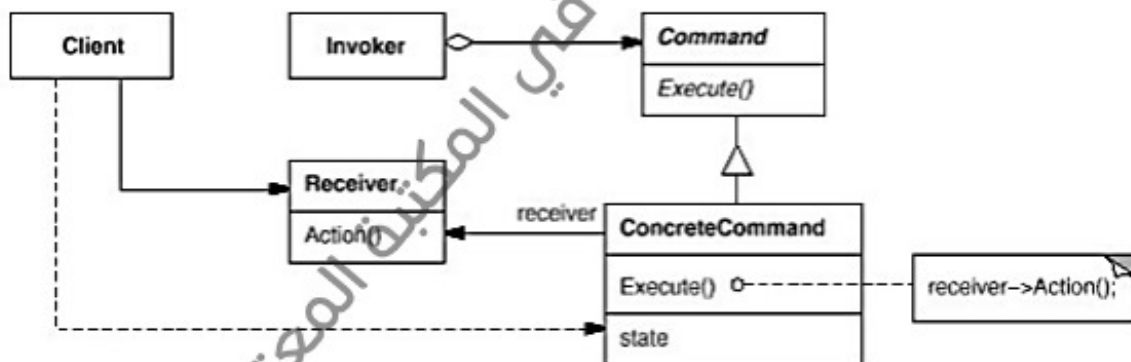
هذه هي فلسفة البرمجة غرضية التوجه طالما نقوم بعملية تجميع فنجمع. ونستخدم abstract method لنعطي التنفيذ للصفوف الأبناء (لا نعيد كتابة شيء).

النوع	الكمية	التكلفة	المحددات
نهرى	وزن	مسافة x كمية	كمية => 300,000
بري	وزن	مسافة x كمية x 4	كمية => 38,000
جوي	حجم	مسافة x كمية x 10	كمية => 80,000
جوي مستعجل	حجم	كلفة الحاوية الجوية x 2	كمية => 80,000

Command

يقوم بتغليف ال request على شكل object، مما يسمح لنا بإرسال ال request للتوابع ك arguments، وتأخير تنفيذ ال request ووضعها في queue، و عمل log لل requests، بالإضافة إلى دعم عمليات غير قابلة للتنفيذ دونها.

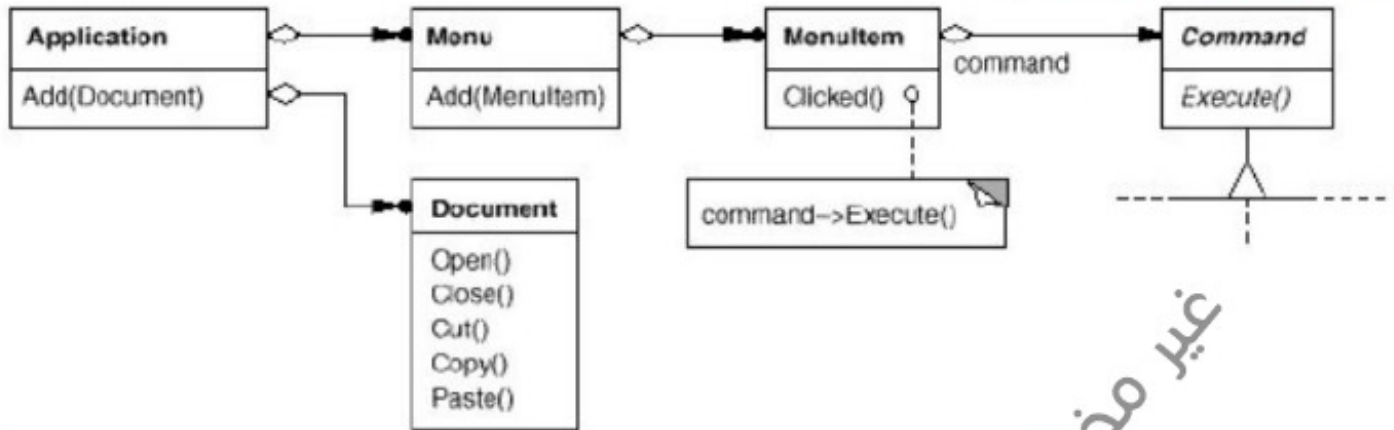
بنية ال command:



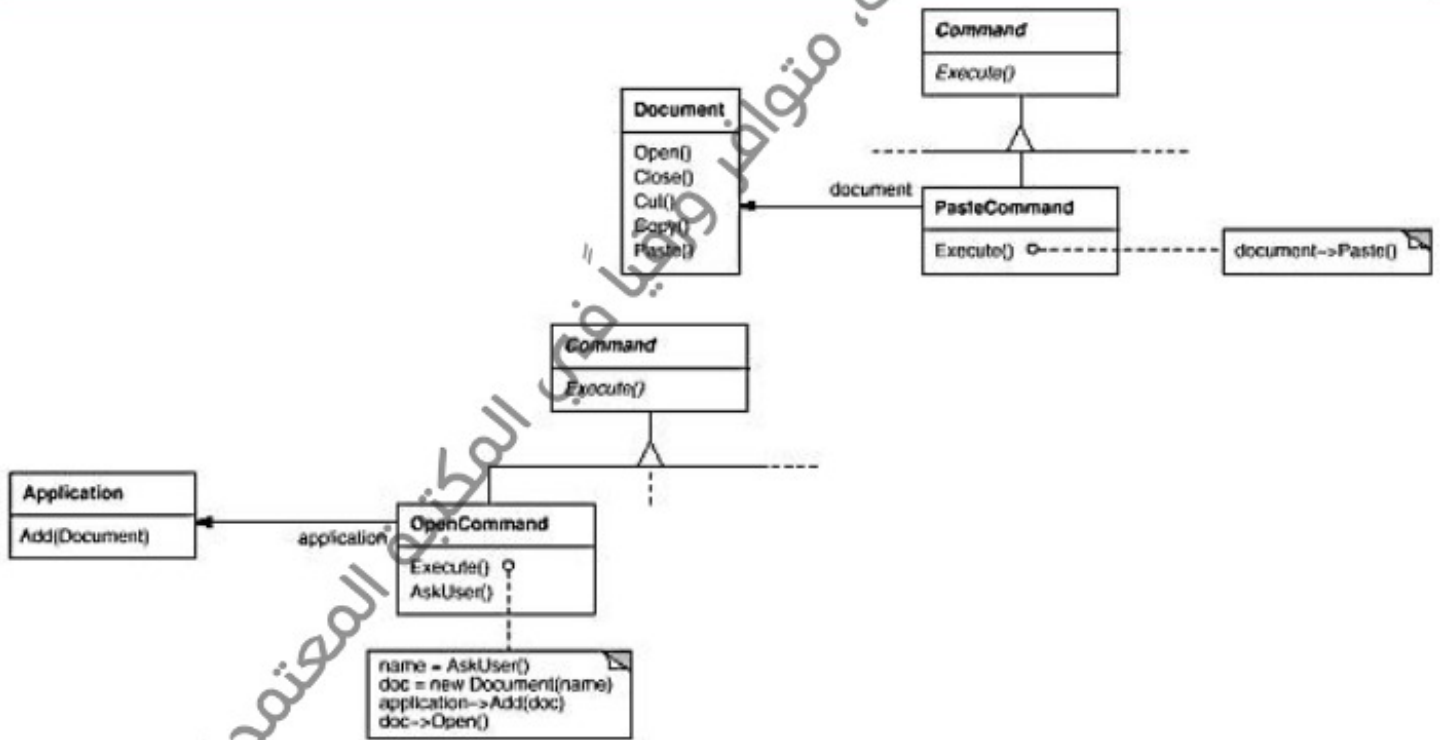
شرح بنية ال command

- يوجد في كلاس command تابع execute() (ممكن أن تكون إضافة أو حذف أو تلوين زر، إلخ...).
- Concrete command يرث من command ويقوم بتنفيذ الأوامر الموجهة له عن طريق التابع Execute (نفس التابع موجود في command ولكن هناك تغيير في سلوكه).
- كما يحتوي concrete command على state (الحالة الحالية للنظام).
- يمكن وجود أكثر من concrete command حسب طبيعة المسألة.
- ال command يمثل صلة الوصل بين مستدعي الطلب (invoker) ومن ينفذه (receiver).

مثال : Menu management :



لدينا Document ويمكن إجراء بعض العمليات عليه (فتح، إغلاق، قص، نسخ، لصق).
 لتنفيذ ذلك على الحاسب نضغط بالزر الأيمن للفأرة على هذا ال document ويظهر لنا Menu تحتوي على
 الإجراءات (فتح، إغلاق، قص، نسخ، لصق).
 عند الضغط على "فتح" أو "لصق" ينفذ `Execute()` الخاص ب Concrete class والذي يمكن أن يكون
`pasteCommand` أو `openCommand` كما في الشكل التالي:



ما ذُلِّل الصَّعْب، ولا لَانَ الحَدِيد، ولا اسْتَقَامَ المَائِل،
 إِلَّا بَعْدَ طَرَفَاتٍ مُسْتَمِرَّةٍ، وَضَرْبَاتٍ دَائِمَةٍ، وَخَطَوَاتٍ
 مُتتَابِعَةٍ
 ثُمَّ تَكُونُ النَتِيجَةُ



```

public interface Command {
    public boolean execute();
    public String getHelpMsg();
}

public class Menu implements Command {
    private HashMap<String,Command> commands = new HashMap<String,Command>();
    public Menu () {
        this.addCommand("help",this);
    }
    private void addCommand(String key, Command command) {
        // inversion of control
        commands.put(key,command);
    }
    public void act() { System.out.println("what do you want to do (typing 'help' can use) ?");
        commands.get(Input.readString()).execute();
    }
    public void execute() {
        // foreach keys and call getHelpMsg on the values
    }
    public String getHelpMsg() { return "display this help"; }
}

public class SaveCommand implements Command {
    public void execute() { (...)}
    public String getHelpMsg() { return "saving ..." }
}

public class LoadCommand implements Command {
    public void execute() { (...)}
    public String getHelpMsg() { return "loading ..." }
}
    
```

يستخدم الـ Command عندما:

1. OO Manner to realize callback functions:

أي في البرمجة الغرضية عند النقر على زر إضافة أو حذف يقوم بعمل callback التابع أو عدة توابع (التابع يمثل Receiver في بنية الـ Command).

2. Queue of requests + Command Object can have a lifetime independent of the original request + we can transfer Command Objects to another space of addresses:

عند خلق object من Command يتم وضع الطلبات في طابور ريثما يتم معالجتها أو نقلها إلى عنوان آخر.

3. Undo operation : Command (with unexecute operation + state) + history stores executed commands:

يمكن التراجع عن عمليات معينة مخزنة مسبقاً ضمن history أو Queue.

4. Recovering from crash = re execute the stored commands:

عند حصول مشكلة ما يمكن إعادة تنفيذ كافة العمليات الموجودة في history أو Queue.

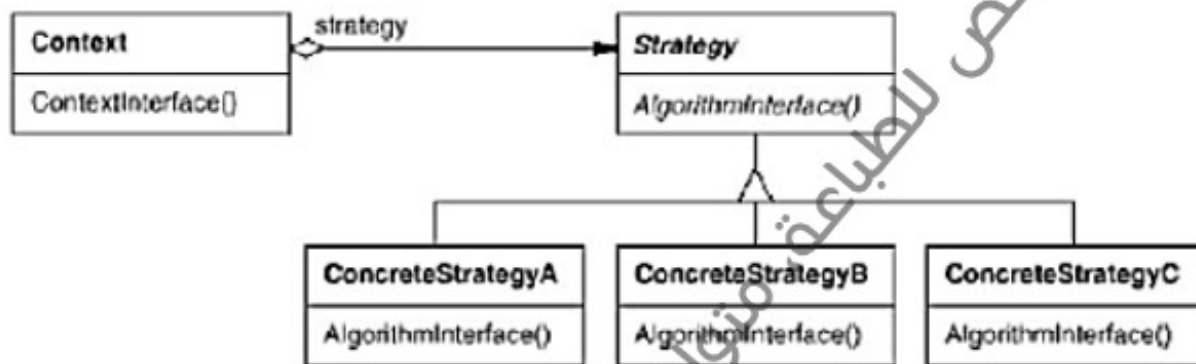
5. Build system of transactions (set of changes on data):

يمكن من خلال ال Command بناء نظام مكون من عدة transactions.

ال transactions ليس فقط العمليات التي تطبق على database وإنما تشمل مجموعة التعليمات في النظام التي قد تنفذ معاً أو لا.

Strategy

- يُعرف عائلة من الخوارزميات ويقوم بتغليف كل منها، مما يجعلها قابلة للتبديل فيما بينها.
- يسمح ال Strategy بتغيير الخوارزمية بشكل منفصل عن ال Client الذي يستخدمها.

**مثال:** لعبة ورقة - حجر - مقص

تلعب هذه اللعبة بلاعبين اثنين وفي كل دورة يلعب كلا اللاعبين إحدى الخيارات التالية : حجر أو ورقة أو مقص. نتيجة الدورة تحسب بالشكل التالي:

-إذا كان اللاعبان يلعبان نفس الخيار فالنتيجة تعادل وكلاهما يحصلان على نقطة.

-الحجر يكسر المقص ومن لعب الحجر يحصل على نقطتين.

-الورقة تغلف الحجر ومن لعب الورقة يحصل على نقطتين.

-المقص يقص الورقة ومن لعب المقص يحصل على نقطتين.

اللعبة تلعب كسلسلة من الدورات وبعدد محدد منذ البداية، الرابع هو من يحصل على أكبر عدد من النقاط بعد انتهاء كل الدورات .

نرغب بإمكانية اللعب مع الحاسب حيث أن عدة استراتيجيات لعب ممكنة:

-دائماً نلعب نفس الخيار.

-نلعب بشكل عشوائي.

-نستخدم لوحة المفاتيح لتحديد خيارنا.

-.....إلخ.



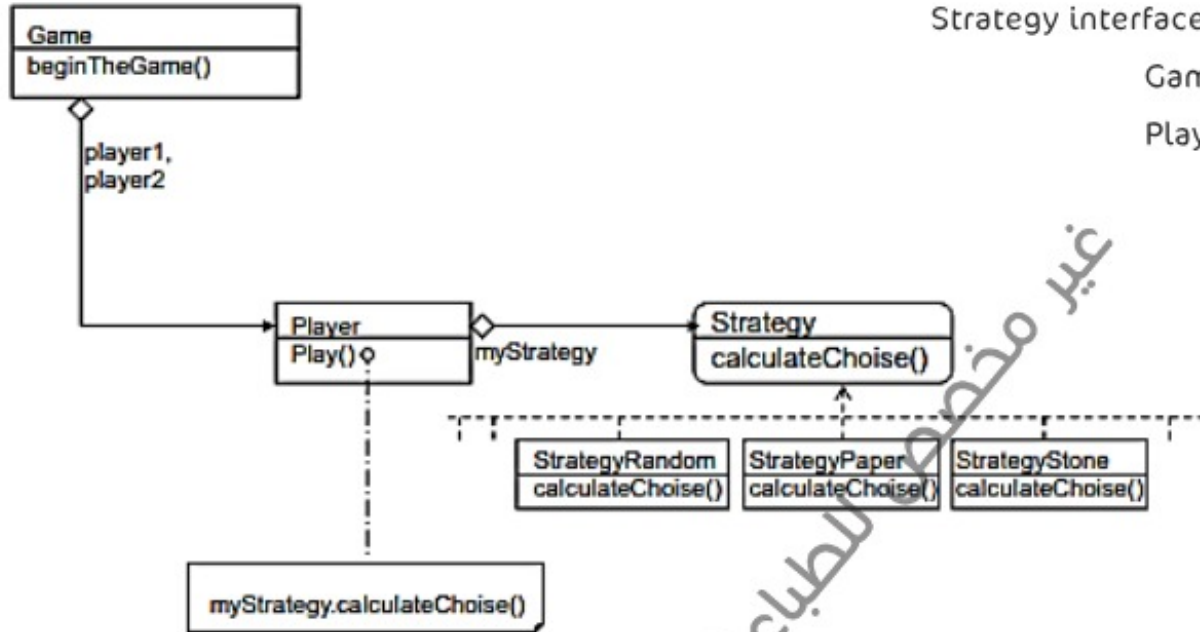
اكتبوا ما يلزم لتحقيق هذه اللعبة:

أنماط التعداد لخيارات اللعب

الواجهة Strategy interface

الصف Game

الصف Player

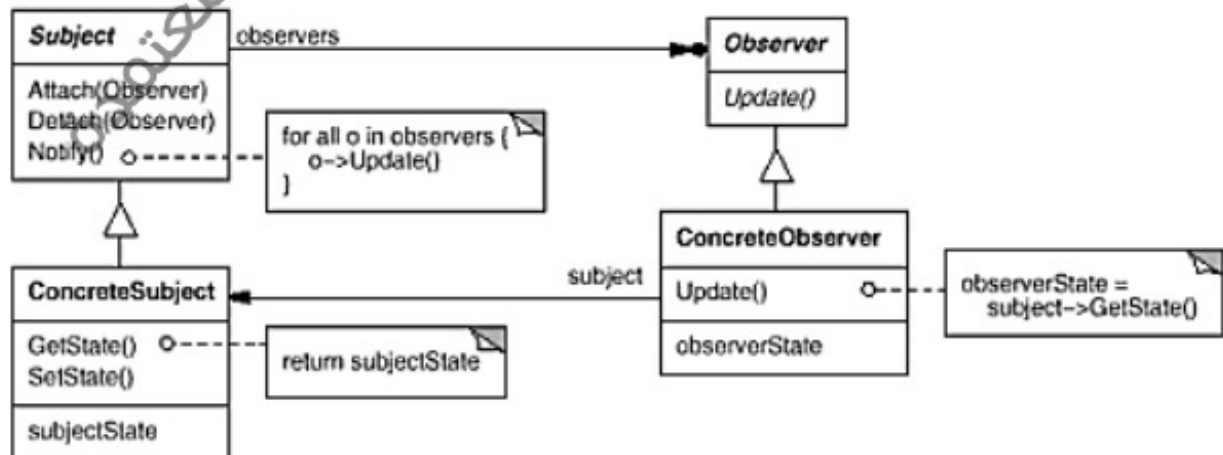


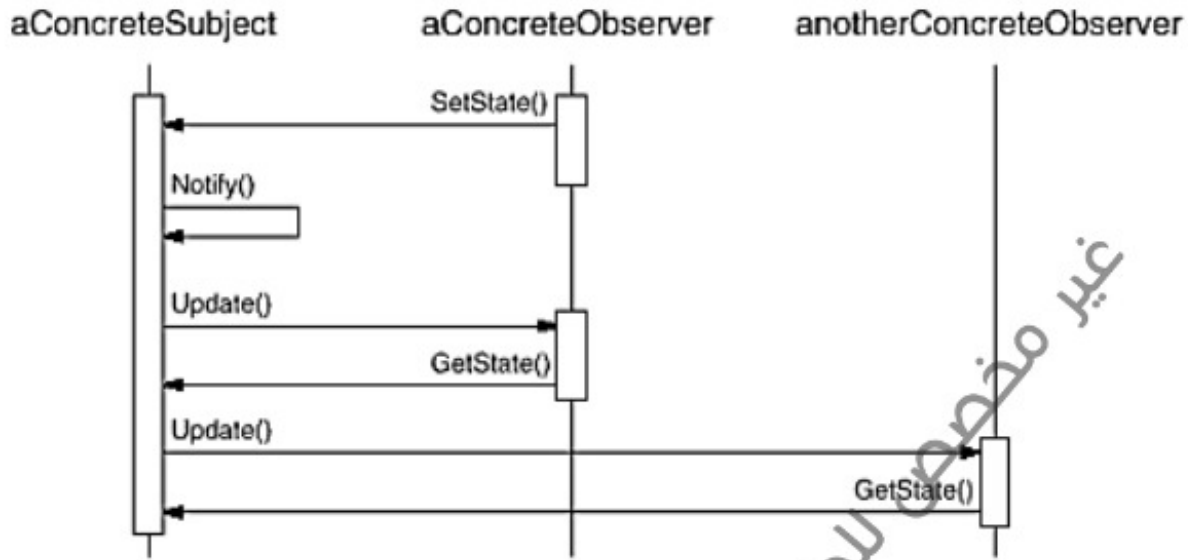
Observer/Subject

- تعريف : هو نمط مستخدم في لغات البرمجة لمراقبة العمليات والأحداث.
- المشكلة:

- غرض (actor) يجب أن يعمل عندما يلاحظ غرض آخر حدث ما.
- لكن ليس بالضرورة أن يعي هذا الغرض بوجود ال actor أو ال actors الآخرين.
- ال actors يمكنهم:
 - أن يكونوا من طبيعة مختلفة.
 - أن يتغيروا دون أن يشعر مصدر الحدث بذلك.

Observer/Subject: Structure



Observer/Subject: Collaborationالمحددات:

- عدة أغراض يمكن أن يتم إنذارهم بالأحداث المنبعثة من مصدر ما.
- عدد و طبيعة الأغراض المنذرين ليس بالضرورة معروف عند زمن الترجمة ويمكن لهم أن يتغيروا مع الزمن.
- مصدر الحدث ومستقبله غير مرتبطين بشكل قوي.
- نحتاج لآلية توكيل بين منبع الحدث (event generator) والمستقبل (listener).

سيناريو

- عندما يرن هاتف فإن عدداً من الأشخاص أو الآلات يمكن أن يكونوا معنيين بالاتصال.
- إن الأشخاص يمكنهم أن يدخلوا أو يخرجوا من الغرفة وبالتالي الدخول والخروج من حقل الانتفاع من الهاتف.

مراحل التصميم بالجافا

- تعريف صفوف الأحداث.
- تعريف واجهات ال listener والصفوف ال adapters (خيارياً).
- تعريف الصف المصدر للأحداث event generator.
- تعريف الصفوف المستقبلية للأحداث listener.

المرحلة الأولى : تعريف صفوف الأحداث

المرحلة الأولى : تعريف صفوف الأحداث

- تعريف صف بنوع الأحداث والذي يمكن إصداره من قبل الـ event generator.
- جعل هذه الصفوف ترث من الصف java.util.EventObject.
- تصميم صفوف الأحداث بحيث يضم الحدث كل المعلومات الضرورية الواجب إرسالها إلى الـ Observer.
- إعطاء صف الحدث اسماً من الصيغة XXXEvent.

```
public class TelephoneEvent extends java.util.EventObject {
    public TelephoneEvent(Telephone source) {
        super(source);
    }
}
```

- EventObject.getSource() يعيد مصدر الحدث ويسمح بالانتساب لعدة مصادر لنفس نوع الحدث.
- لدينا نمطان من الـ Observer pattern:
 1. Pull model حيث ان الـ Observer يستخرج المعلومات من الـ Source.
 2. Push model حيث أن كل المعلومات أصلاً مرسلة مع الحدث.

المرحلة الثانية تعريف واجهات الـ Listener

- نعرف لكل نوع من الأحداث واجهة ترث من java.util.EventListener و تحوي من أجل كل حدث على منهج والذي سيتم استدعاؤه عند الإعلام (notification) بوقوع حدث.
- اسم هذه الواجهة هو اسم صف الحدث مع استبدال Event بـ Listener.
- أسماء مناهج الواجهة تبني من فعل ماض يدل على حالة الـ activation.
- كل منهج يعيد void ويأخذ وسيطاً من صف الحدث.

```
public interface TelephoneListener extends
    java.util.EventListener {
    public void telephoneRang(TelephoneEvent e);
    public void telephoneAnswered(TelephoneEvent e);
}
```

المرحلة الثالثة: تعريف الصفوف المصدرة للأحداث

- نعرف لكل نوع من الأحداث زوج من المناهج add/remove
- أسماء المناهج الانتساب وفك الانتساب هي من الصيغة التالية:
 - Addlistener-interface-name() ✓
 - Removelistener-interface-name() ✓
- ولكل منهج من واجهة ال listener فإننا نعرف منهج private لانتشار الحدث، هذا المنهج بلا وسطاء ويعيد .void
- هذا المنهج نسميه بالصيغة التالية:
 - Firelistener-method-name()
- إطلاق الحدث.

```

import java.util.*;
public class Telephone {
private ArrayList<TelephoneListener> telephoneListeners = new ArrayList<TelephoneListener>();
public void ringPhone() {
fireTelephoneRang();
}
public void answerPhone() {
fireTelephoneAnswered();
}
public synchronized void addTelephoneListener(TelephoneListener l) {
if (telephoneListeners.contains(l)) return ; }
telephoneListeners.add(l);
}
public synchronized void removeTelephoneListener(TelephoneListener l){
telephoneListeners.remove(l);
}
private void fireTelephoneRang() {
ArrayList<TelephoneListener> tl = (ArrayList<TelephoneListener>) telephoneListeners.clone();
if (tl.size() == 0) { return; }
TelephoneEvent event = new TelephoneEvent(this);
for (TelephoneListener listener : tl) {
listener.telephoneRang(event);
}
}
private void fireTelephoneAnswered() {... }
}

```

المرحلة الرابعة: تعريف الصفوف المتسلسلة

هذه الصفوف ما عليها الا أن تنفذ ال Listeners المرتبطة بها:

```

public class AnsweringMachine implements TelephoneListener {
    public void telephoneRang(TelephoneEvent e) {
        System.out.println("AM hears the phone ringing.");
    }
    public void telephoneAnswered(TelephoneEvent e) {
        System.out.println("AM sees that the phone was answered.");
    }
}
class MyTelephoneAdapter extends TelephoneAdapter {
    public void telephoneRang(TelephoneEvent e) {
        System.out.println("I'll get it!");
    }
}
public class Person {
    public void listenToPhone(Telephone t) {
        t.addTelephoneListener(new MyTelephoneAdapter());
    }
}

```

الاستخدام:

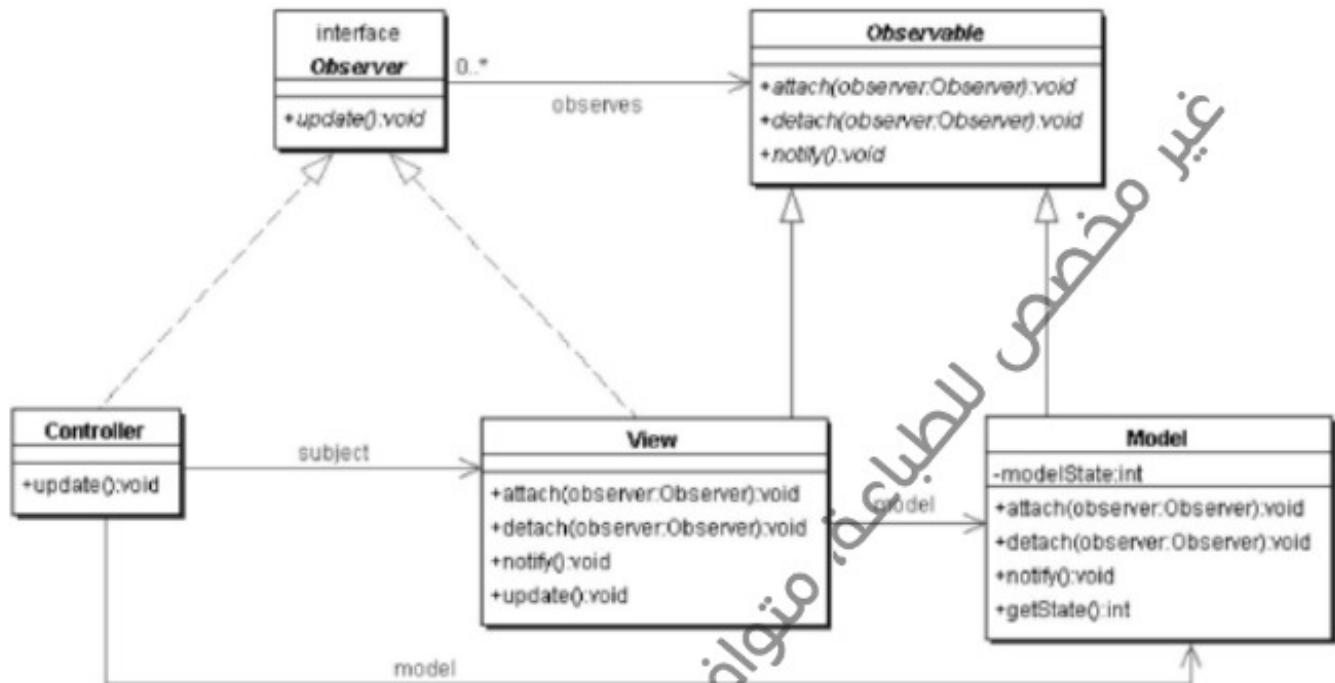
```

public class Example1 {
    public static void main(String[] args) {
        Telephone ph = new Telephone();
        AnsweringMachine am = new AnsweringMachine();
        Person bob = new Person();
        ph.addTelephoneListener(am);
        bob.listenToPhone(ph);
        ph.ringPhone();
        ph.answerPhone();
    }
}

```

Observer/Subject: MVC

يُعرف علاقة one to many بين الأغراض بحيث أن حصول تغيير في حالة غرض معين (state) يُسبب تنبيه وتحديث تلقائي لجميع الأغراض التابعة له.



شرح رسمة observe/Subject MVC:

- يمثل ال Controller ال observer لأنه يراقب التعديلات التي قد تطرأ على ال Model (يمكن تشبهه ال controller ب Listener).
- يمثل ال Model ال Observable أي هو مُراقب من قبل ال Controller (يمكن تشبيهه ال Model كمنشئ للحدث Event).
- View عبارة عن observable و observer لأنه يراقب ال Model ويستجيب للمتغيرات التي طرأت على ال Model وكذلك هو observable لأنه مراقب من قبل ال Controller (أي ال view هو Listener و event).

إلى هنا نكون قد انتهينا من Design pattern

بأنواعه يرجى دراسة تمارين أكثر والتدرب عليها.

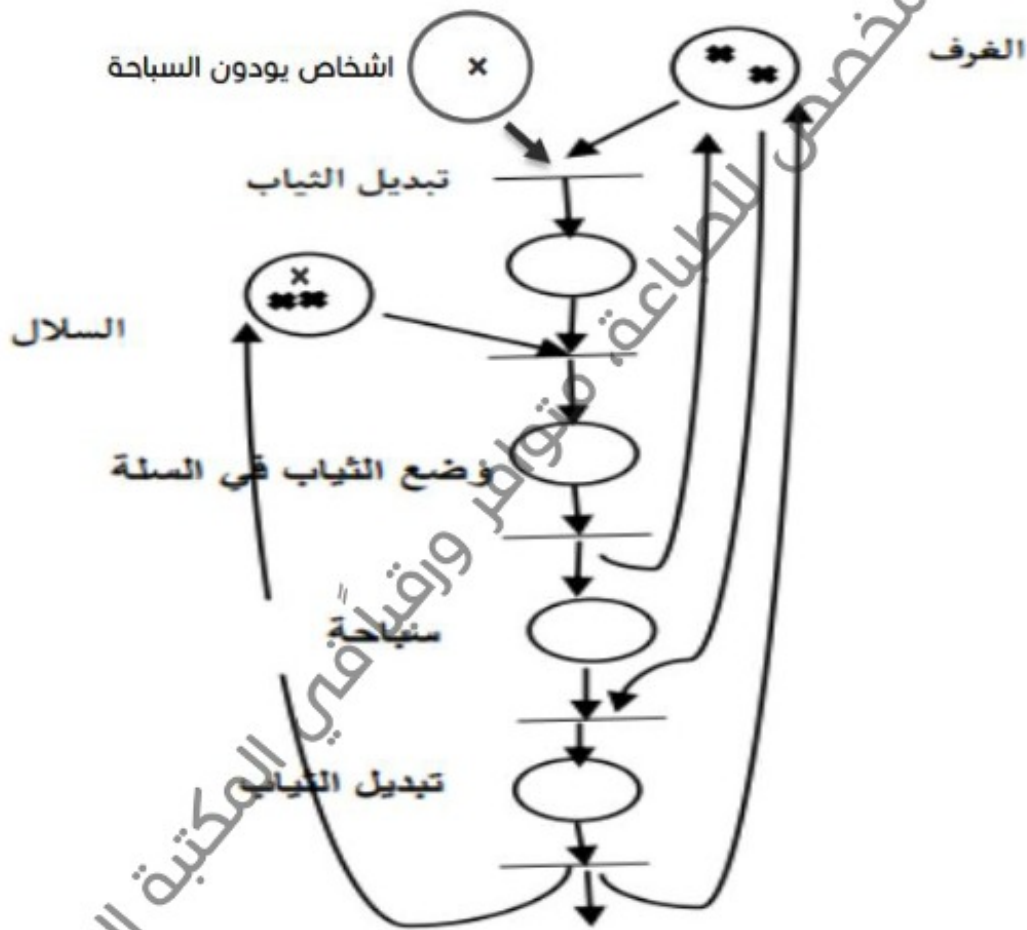
Petri Net (شبكات بتري)

أداة مرئية رياضية تسمح بنمذجة سلوك النظم المتوازية (التسايرية) والتحقق من سلوكها.

تم شرح الفرق بين verification & validation في المحاضرة الأولى وشرح مفاهيم شبكات بتري و محدوداتها.

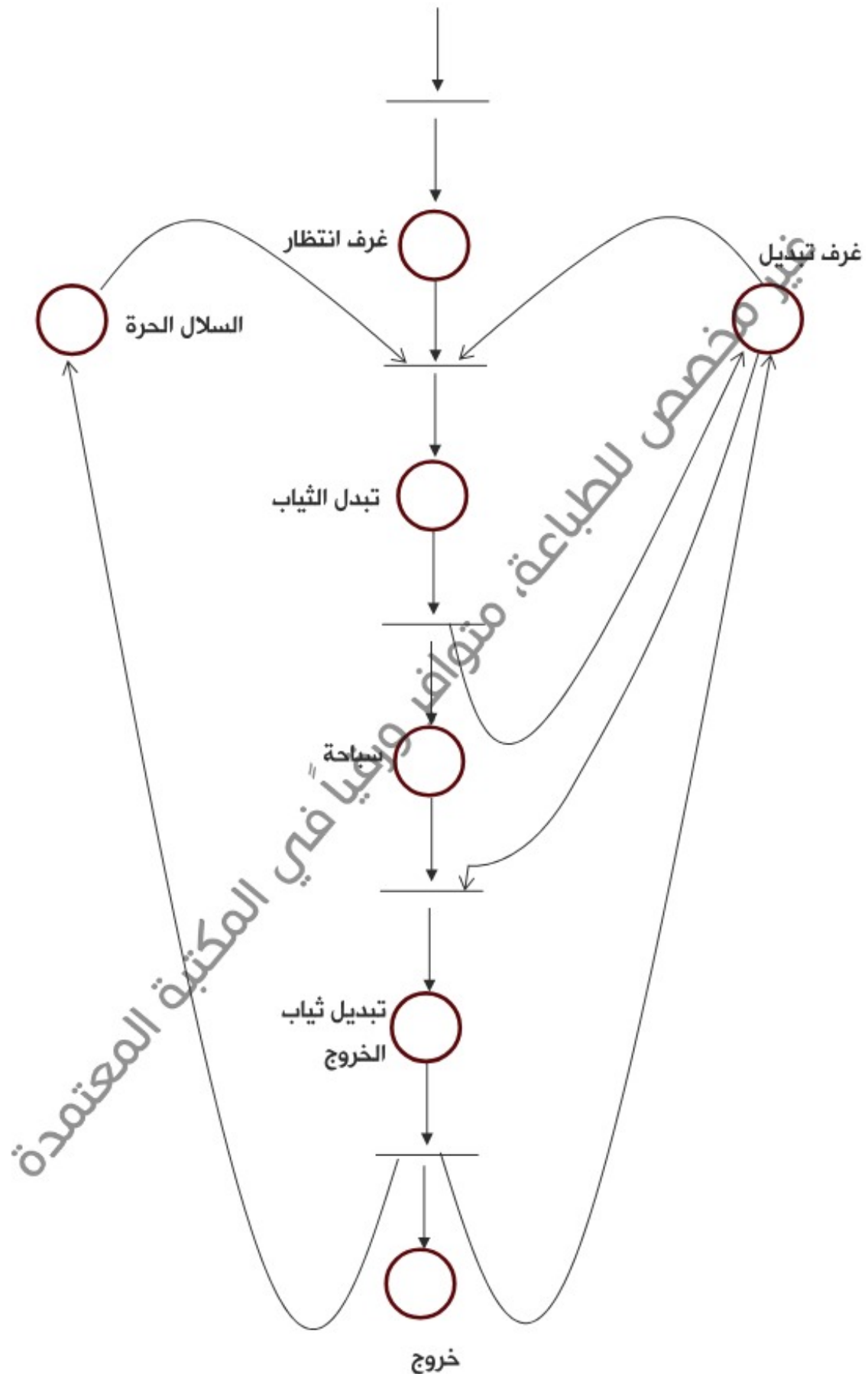
مثال : بروتوكول إدارة مسبح:

تم شرح هذا المثال في المحاضرة الأولى وكانت شبكة بتري لهذا البروتوكول:



وقد شرحنا كيف أن هذا النظام سيسبب deadlock، ولحل هذه المشكلة نقوم بإعادة نمذجة النظام (إعادة

رسم شبكة بتري):

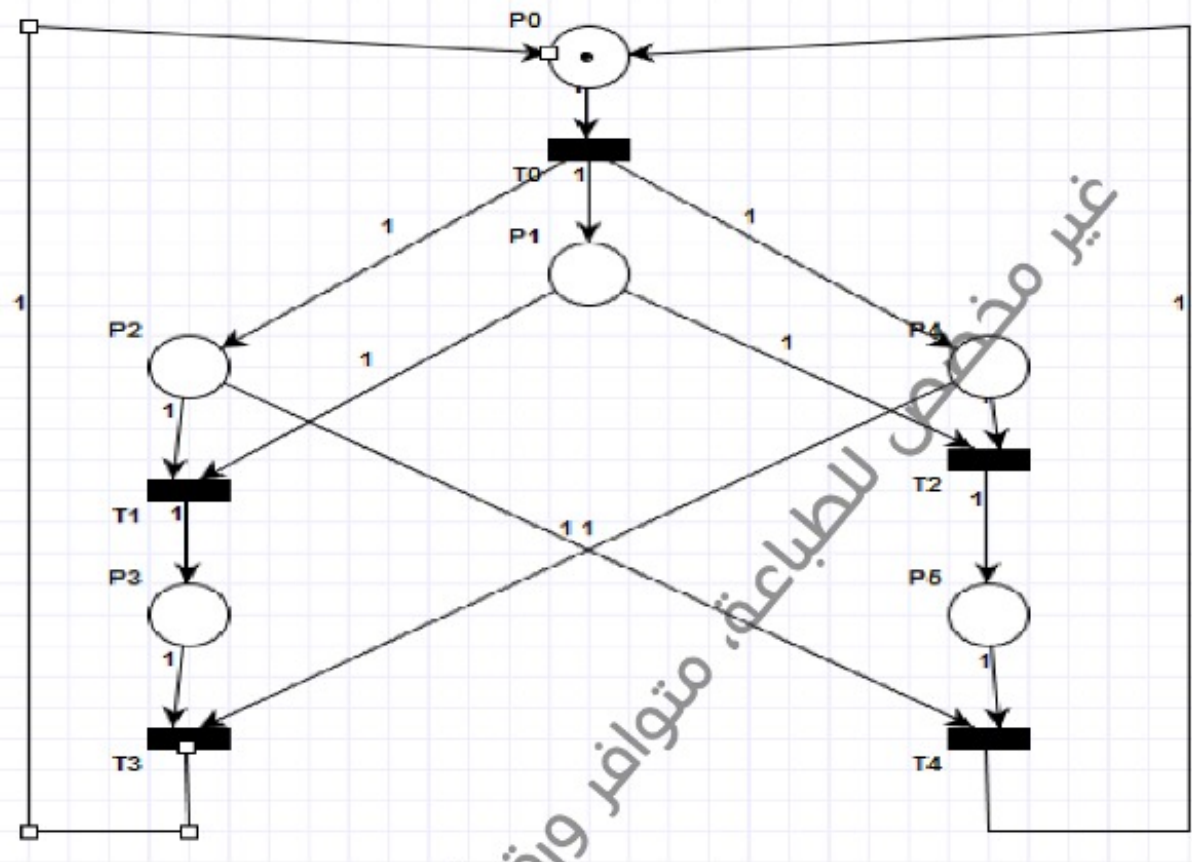


وبذلك نضمن أن النظام لن يتوقف ولن تحدث حالة اختناق Deadlock

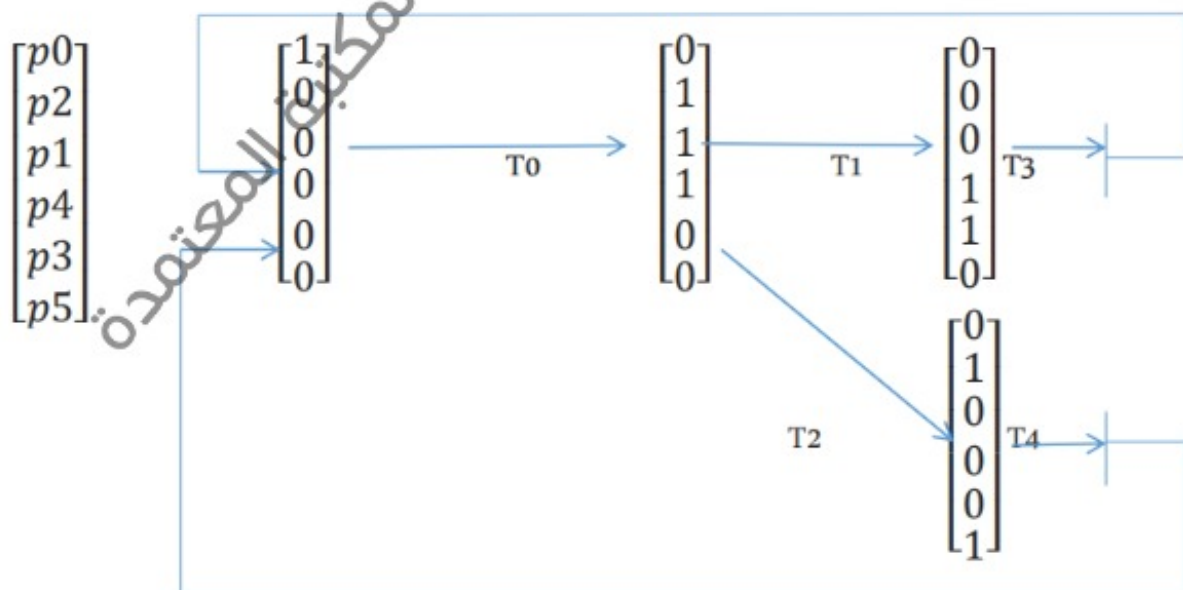
الموديل الرياضي لشبكة بترى نت:

أشعة الترقيم و أشجار التغطية Mark Vectors & Reachability Graphs

بفرض لدينا الشبكة لتالية:



ارسم شجرة التغطية للمثال السابق:



- بكل مكان من الأماكن أضع عدد العلامات الموجودة.
- النظام السابق هو نظام حي لأنه يعيد نفسه (لا يوجد فيه توقف)، ممكن أن يكون حي جزئياً بتوقف إحدى الحالات ولكن يكون هناك إحدى الحلقات تستمر.
- الشرط المرفوض لا يمكن للنظام أن يمر فيه.
- الشرط المطلوب من النظام لا يمكن تجاوزه.
- يكون النظام فيه Deadlock إذا كان هناك شعاع لا يوجد فيه انتقال.

شبكات بتري تساعد في معرفة إن كان هناك توقف في النظام أم لا، وتساعد في كيفية استثمار الموارد بشكل أمثل وسرعة قصوى.

تم شرح نظم الزمن الحقيقي ومثال المنجم في المحاضرة الأولى.

في المحاضرات القادمة سنأخذ أمثلة أكثر على شبكات بتري وأنواعها و خوارزمية شجرة التغطية ونظم الزمن الحقيقي.



النهاية